



## Spring 2023, Lab Manual – 05

### LLO 02: React Js

#### **Contents:**

- Intro to Web Engineering
- Technologies
- Tools
- Introduction to REACT
- Getting Started
- The Basics
- Introduction to JSX
- React Components
- React Props
- Unilateral Data Flow
- Task: Create A Dropdown Button Filter

#### Introduction to Web Engineering

Web Engineering is the application of systematic and quantifiable approaches (concepts methods, techniques tools) to cost - effective requirements analysis, design, implementation, testing, operation, and maintenance of high quality Web applications.

#### Technologies to be studied

- HTML
- CSS
- JavaScript
- Bootstrap
- JQuery
- PHP
- MySQL [Database]
- Laravel [PHP FRAMEWORK]
- REACT JS

## Tools – IDEs

- Visual Studio Code
- Adobe Dreamweaver
- Visual Studio

### 10.1 Introduction to REACT:

React is a UI library developed at Facebook to facilitate the creation of interactive, stateful & reusable UI components. As Wikipedia puts it, React allows developers to “create large web-applications that use data and can change over time without reloading the page.”

When JavaScript interacts with an HTML document, it is interacting with the Document Object Model, or DOM, which is a tree containing all of the HTML elements of that document. React uses a concept called the Virtual DOM to selectively render nodes on this tree. In essence, this allows React to interact with as little of the DOM as possible while still adequately making changes to the state of a webpage.

As an example, let's pretend that your website is the full body rendering of a person, from head to toe. The DOM would say “on the FACE, there are the EYES, NOSE, EARS, etc.”. But, what happens if you want to change a feature on the person's body, like give the person some beautiful Steve Buscemi eyes. Normally, the person will completely be re-rendered with the new eyes. However, in React-land, when we apply these changes, two things take place. First, React runs a “diffing” algorithm, which identifies what has changed. The second step is reconciliation, where it updates the DOM with the results of the diff (i.e. changing the eyes to Steve's).

If the idea of the Virtual DOM is confusing, don't worry! It's difficult to wrap one's head around, and knowing exactly how the DOM and Virtual DOM function aren't necessary for this lab. Just know that React is used because it optimizes DOM interaction.

We have already studied DOM in previous lab, to know more about DOM is and how it functions, [this link](#) is very helpful.

### 10.2 Getting Started:

1. Check that you have node installed by running `node --version` in your terminal 2. If you don't have node you can install it [here](#); get the appropriate version for your computer

2. Create a folder and open it with vs code and then in the terminal run this command  
npm create vite@latest your-project-name --template react
3. Vite supports various templates beyond React. You can specify different frameworks or libraries you can move down and select as per instructions
4. Then run cd my-app to go into that directory
7. Run npm run dev to start your app
8. In your browser, go to address in the terminal to see your app live!
9. Now open up any text editor of your choice and open the my-app folder
10. All of the changes that we will be making will be in the src directory

## 10.3 The Basics:

React's basic building blocks are called components. Let's write one. Create a new file in the src directory called HelloWorld.jsx. Add this code inside:

```
import React, { Component } from 'react';

class HelloWorld extends Component {render() {

  return (

    <h1>Hello World!</h1>

  );
}

}

export default HelloWorld;
```

Change App.js to look like this:

```
import React, { Component } from 'react';import './App.css';

import HelloWorld from './HelloWorld';

class App extends Component {
  render() {

    return (

      <div className="App">

        <HelloWorld />

      </div>

    );
  }
}
```

```
}
```

After saving your changes, you can view them in your localhost tab without needing to re

```
}
```

run from the command line.

```
export default App;
```

If you haven't seen this syntax before, you are probably wondering what Javascript/HTML chimera sorcery is taking place right now.

## 10.4 Introduction to JSX:

This so-called sorcery is called JSX, and it is a Javascript XML syntax transform. This lets you write HTML-ish tags in your Javascript. Note that this is not exact HTML—you are really just writing XML-based object representations.

For regular html tags, the class attribute is `className` and the for attribute is `htmlFor` in JSX because these are reserved words in Javascript. A more in-depth explanation of JSX can be found [here](#). While you can certainly use React without JSX, we highly recommend that you use JSX.

## Difference in HTML and JSX:

In HTML we use the `class` attribute. It's probably the most widely used attribute, for various reasons. One of those reasons is CSS. The `class` attribute allows us to style HTML elements easily, and CSS frameworks like Tailwind put this attribute to the center of the CSS user interface design process. But there's a problem. We are writing this UI code in a JavaScript file, and `class` in the JavaScript programming language is a reserved word. This means we can't use this reserved word as we want. It serves a specific purpose (defining JavaScript classes) and the React creators had to choose a different name for it. That's how we ended up with `className` instead of `class`.

## 10.5 REACT Components:

React components are independent, reusable classes that compose different parts of your UI. React applications are made up of components, many of which are rendered within other components.

It is generally a good idea to write each component in its own file. For example, the `HelloWorld` component is implemented in `HelloWorld.jsx`. To use it elsewhere, simply import the component at the top of the file that you wish to use it in, as we did at the top of `App.js`. Note that you must export the component at the bottom of its own file in order to be able to import it elsewhere.

We're going to create a `WelcomeMessage` component:

```
function WelcomeMessage() {  
  
  return <p>Welcome!</p>  
  
}
```

See? It's a simple function that returns a line of JSX that represents a p HTML element. We're going to add it to the App.js file

## 10.6 REACT Props:

In React.js, props (short for "properties") are a mechanism for passing data from a parent component to a child component. Think of them as arguments you can pass to a function, but in this case, they're arguments you pass to a React component

This is a functional component and here props is passed.

```
function WelcomeMessage(props) {  
  return <p>Welcome!</p>  
}
```

In app.jsx we can pass props to the child component as:

```
<WelcomeMessage myprop={'somevalue'} />
```

## Lifecycle Methods

The render() method is the only required method for creating a component, but there are several lifecycle methods and specs we can use that can be helpful which you can read about [here](#).

**React Hooks:** When you call the hook, you pass it a function. The function will be run by React when the component is first rendered, and on every subsequent rerender/update.

## State:

Every component has a state object and a props object. Initial state should be set in the constructor(), but can be set or reset elsewhere using the setState() method. Calling setState() triggers UI updates and is the bread and butter of React's interactivity. Let's create try this by implementing a Counter component in a new file, Counter.jsx:

```
import React, { Component } from 'react';
```

```
class Counter extends  
  Component {  
  constructor(props) {  
  
    super(props);  
  
    this.state = {  
      count: 5  
    };  
  }  
}
```

```
  render() {  
    return (  
  
      <h1>{this.state.count}</h1>  
  
    );  
  }  
}
```

```
export default Counter;
```

Think about a form, for example. Each individual input element of the form is responsible for managing its state: what is written inside it. A button is responsible for knowing if it's being clicked, or not. If it's on focus. A link is responsible for knowing if the mouse is hovering it. In React, or in any other components-based framework/library, all our applications are based and make heavy use of components state. We manage state using the useState utility provided by React

You import `useState` from React in this way:

```
import React, { useState } from 'react'
```

`useState()` accepts the initial value of the state item and returns an array containing the state variable, and the function you call to alter the state.

```
import { useState } from 'react'

const Counter = () => {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click</button>
    </div>
  )
}

ReactDOM.render(<Counter />, document.getElementById('root'))
```

The `useEffect` hook allows components to have access to the lifecycle events of a component. When you call the hook, you pass it a function. The function will be run by React when the component is first rendered, and on every subsequent rerender/update. React first updates the DOM, then calls any function passed to `useEffect()`.

```

const { useEffect, useState } = React

const CounterWithNameAndSideEffect = () => {
  const [count, setCount] = useState(0)

  useEffect(() => {
    console.log(`You clicked ${count} times`)
  })

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click</button>
    </div>
  )
}

```

## Events:

React also has a built in cross browser events system. The events are attached as properties of components and can trigger methods. Let's make our count increment below using events. You can use `this.setState()` to modify state.

```
import React, { Component } from 'react';
```

```
class Counter extends Component { constructor(props) {
  super(props);
```

```
  this.state = { count: 5
};
}
```

```
  incrementCount = () => {
    this.setState({
      count: this.state.count + 1
    })
  }
}
```

```

});
}

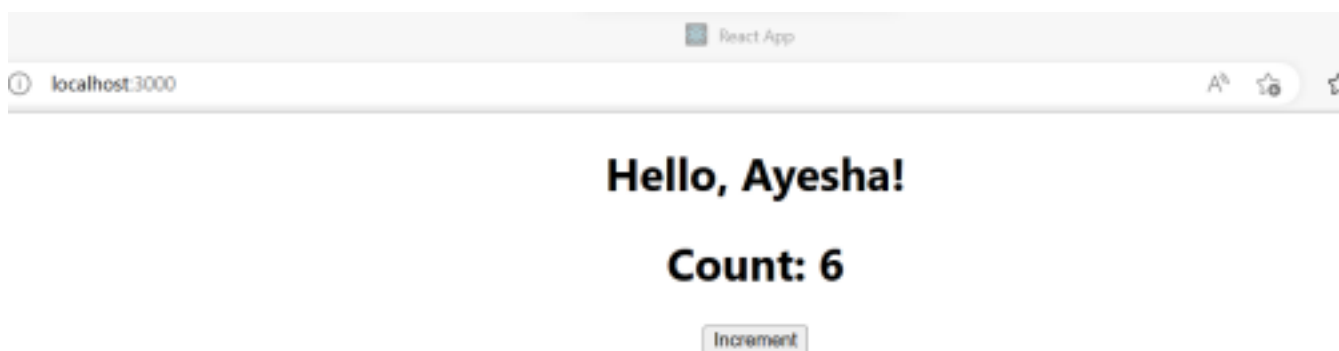
render() {
  return (
    <div>
      <h1>Count: {this.state.count}</h1>
      <button onClick={this.incrementCount}>Increment</button>
    </div>

  );
}
}

export default Counter;

```

Once you've completed your Counter, check through browser. It should look something like this.



## 10.7 React Routing:

React routing is the process of managing navigation and displaying different components based on the URL in a React application. It allows you to create single-page applications (SPAs) that feel like traditional multi-page websites, but without full page reloads.

How Routing Works:

1. URL Changes: When the user clicks a Link component or manually changes the URL,

the BrowserRouter detects the change.

2. Route Matching: The Routes component checks the current URL against the path props of the Route components.
3. Component Rendering: When a match is found, the component specified in the element prop of the matching Route is rendered.
4. No Page Reloads: The content of the page is updated dynamically without a full page reload.

Working with Routes :

1. Install Router:

```
npm install react-router-dom
```

2. Import React Router Components

```
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
```

We wrap our content first with `<BrowserRouter>`.

Then we define our `<Routes>`. An application can have multiple `<Routes>`. Our basic example only uses one.

`<Route>`s can be nested. The first `<Route>` has a path of `/` and renders the Layout component.

The nested `<Route>`s inherit and add to the parent route. So the blogs path is combined with the parent and becomes `/blogs`.

```
import React, { useRef, useEffect } from 'react';

import { BrowserRouter as Router, Routes, Route, useLocation } from 'react-router-dom';

import Counter from './components/counter';

import CounterWithVariable from './components/CounterWithVariable.jsx';

import PropsComponent from './components/PropsComponent';

import Navbar from './components/Navbar.jsx';

import Home from './components/Home.jsx';

import About from './components/About.jsx';

import Contact from './components/Contact.jsx';
```

```
import './components/Navbar.css';

import Form from './components/Form.jsx';

import './components/Form.css';

function App() {

  return (

    <Router>

      <div>

        <Navbar />

        <Routes>

          <Route path="/" element={<Home />} />

          <Route path="/about" element={<About />} />

          <Route path="/contact" element={<Contact />} />

        </Routes>

        <h1>Counter Example</h1>

        <Counter />

        <h1>Counter Example without use state</h1>

        <CounterWithVariable />

        <h1>Props Example:</h1>

        <PropsComponent name="any name" message="hello!" />

        <h2>Form Handlings</h2>

        <Form />

      </div>

    </Router>

  )
}
```

```
</Router>

);
}

export default App;
```

## Form Validation In React:

Form validation is the process of ensuring that user input in a form meets specific criteria before it's submitted. This is crucial for:

- Data Integrity: Preventing invalid or malicious data from being stored in your database.
- 
- User Experience: Providing clear feedback to users about errors in their input.
- Security: Protecting your application from vulnerabilities like SQL injection or cross-site scripting (XSS).

## Implementation Steps:

### 1. State Management:

```
import React, { useState } from 'react';

function MyForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [nameError, setNameError] = useState("");
  const [emailError, setEmailError] = useState("");
  // ... other state variables
}
```

### 2. Input Handling:

```
<input
  type="text"
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

### 3. Validation Functions

```
const validateName = () => {
  if (!name.trim()) {
    setNameError('Name is required');
    return false;
  } else {
    setNameError('');
    return true;
  }
};
```

```
const validateEmail = () => {
  const emailRegex = /^[S+@\S+\.\S+]/;
  if (!email.trim()) {
    setEmailError('Email is required');
    return false;
  } else if (!emailRegex.test(email)) {
    setEmailError('Invalid email format');
    return false;
  } else {
    setEmailError('');
  }
};
```

```
    return true;  
}  
};
```